# Python - Add Dictionary Items

## Add Dictionary Items

Adding dictionary items in Python refers to inserting new key-value pairs into an existing dictionary. Dictionaries are mutable data structures that store collections of key-value pairs, where each key is associated with a corresponding value.

Adding items to a dictionary allows you to dynamically update and expand its contents as needed during program execution.

We can add dictionary items in Python using various ways such as –

- Using square brackets
- Using the update() method
- Using a comprehension
- Using unpacking
- Using the Union Operator
- Using the |= Operator
- Using setdefault() method
- Using collections.defaultdict() method

## Add Dictionary Item Using Square Brackets

The square brackets **[]** in Python is used to access elements in sequences like lists and strings through indexing and slicing operations. Additionally, when working with dictionaries, square brackets are used to specify keys for accessing or modifying associated values.

You can add items to a dictionary by specifying the key within square brackets and assigning a value to it. If the key is already present in the dictionary object, its value will

be updated to val. If the key is not present in the dictionary, a new key-value pair will be added.

## Example

In this example, we are creating a dictionary named "marks" with keys representing names and their corresponding integer values. Then, we add a new key-value pair 'Kavta': 58 to the dictionary using square bracket notation –

```
marks = {"Savita":67, "Imtiaz":88, "Laxman":91, "David":49}
print ("Initial dictionary: ", marks)
marks['Kavya'] = 58
print ("Dictionary after new addition: ", marks)
```

It will produce the following output –

```
Initial dictionary:  {'Savita': 67, 'Imtiaz': 88, 'Laxman': 91, 'David': 49}
Dictionary after new addition:  {'Savita': 67, 'Imtiaz': 88, 'Laxman': 91, 'Davic
```

# Add Dictionary Item Using the update() Method

The update() method in Python dictionaries is used to merge the contents of another dictionary or an iterable of key-value pairs into the current dictionary. It adds or updates key-value pairs, ensuring that existing keys are updated with new values and new keys are added to the dictionary.

You can add multiple items to a dictionary using the update() method by passing another dictionary or an iterable of key-value pairs.

## Example

In the following example, we use the update() method to add multiple new key-value pairs 'Kavya': 58 and 'Mohan': 98 to the dictionary 'marks' –

```
marks = {"Savita":67, "Imtiaz":88}
print ("Initial dictionary: ", marks)
```

```
marks.update({'Kavya': 58, 'Mohan': 98})
print ("Dictionary after new addition: ", marks)
```

We get the output as shown below −

```
Initial dictionary:  {'Savita': 67, 'Imtiaz': 88}
Dictionary after new addition:  {'Savita': 67, 'Imtiaz': 88, 'Kavya': 58, 'Mohan
```

## Add Dictionary Item Using Unpacking

Unpacking in Python refers to extracting individual elements from a collection, such as a list, tuple, or dictionary, and assigning them to variables in a single statement. This can be done using the * operator for iterables like lists and tuples, and the ** operator for **dictionaries**.

We can add dictionary items using unpacking by combining two or more dictionaries with the ** unpacking operator.

### Example

In the example below, we are initializing two dictionaries named "marks" and "marks1", both containing names and their corresponding integer values. Then, we create a new dictionary "newmarks" by merging "marks" and "marks1" using dictionary unpacking −

```
marks = {"Savita":67, "Imtiaz":88, "Laxman":91, "David":49}
print ("marks dictionary before update: \n", marks)
marks1 = {"Sharad": 51, "Mushtaq": 61, "Laxman": 89}
newmarks = {**marks, **marks1}
print ("marks dictionary after update: \n", newmarks)
```

Following is the output of the above code −

```
marks dictionary before update:
 {'Savita': 67, 'Imtiaz': 88, 'Laxman': 91, 'David': 49}
marks dictionary after update:
 {'Savita': 67, 'Imtiaz': 88, 'Laxman': 89, 'David': 49, 'Sharad': 51, 'Mushtaq'
```

## Add Dictionary Item Using the Union Operator (|)

The union operator in Python, represented by the **|** symbol, is used to combine the elements of two sets into a new set that contains all the unique elements from both sets. It can also be used with dictionaries in Python 3.9 and later to merge the contents of two dictionaries.

We can add dictionary items using the union operator by merging two dictionaries into a new dictionary, which includes all key-value pairs from both dictionaries.

### Example

In this example, we are using the | operator to combine the dictionaries "marks" and "marks1" with "marks1" values taking precedence in case of duplicate keys −

```
marks = {"Savita":67, "Imtiaz":88, "Laxman":91, "David":49}
print ("marks dictionary before update: \n", marks)
marks1 = {"Sharad": 51, "Mushtaq": 61, "Laxman": 89}
newmarks = marks | marks1
print ("marks dictionary after update: \n", newmarks)
```

Output of the above code is as shown below −

```
marks dictionary before update:
 {'Savita': 67, 'Imtiaz': 88, 'Laxman': 91, 'David': 49}
marks dictionary after update:
 {'Savita': 67, 'Imtiaz': 88, 'Laxman': 89, 'David': 49, 'Sharad': 51, 'Mushtaq'
```

## Add Dictionary Item Using the "|=" Operator

The **|=** operator in Python is an in-place union operator for sets and dictionaries. It updates the set or dictionary on the left-hand side with elements from the set or dictionary on the right-hand side.

We can add dictionary items using the |= operator by updating an existing dictionary with key-value pairs from another dictionary. If there are overlapping keys, the values from the right-hand dictionary will overwrite those in the left-hand dictionary.

### Example

In the following example, we use the |= operator to update "marks" with the key-value pairs from "marks1", with values from "marks1" taking precedence in case of duplicate keys −

```
marks = {"Savita":67, "Imtiaz":88, "Laxman":91, "David":49}
print ("marks dictionary before update: \n", marks)
marks1 = {"Sharad": 51, "Mushtaq": 61, "Laxman": 89}
marks |= marks1
print ("marks dictionary after update: \n", marks)
```

The output produced is as shown below −

```
marks dictionary before update:
 {'Savita': 67, 'Imtiaz': 88, 'Laxman': 91, 'David': 49}
marks dictionary after update:
 {'Savita': 67, 'Imtiaz': 88, 'Laxman': 89, 'David': 49, 'Sharad': 51, 'Mushtaq'
```

## Add Dictionary Item Using the setdefault() Method

The setdefault() method in Python is used to get the value of a specified key in a dictionary. If the key does not exist, it inserts the key with a specified default value.

We can add dictionary items using the setdefault() method by specifying a key and a default value.

### Example

In this example, we use the setdefault() to add the key-value pair "major": "Computer Science" to the "student" dictionary −

```
# Initial dictionary
student = {"name": "Alice", "age": 21}
# Adding a new key-value pair
major = student.setdefault("major", "Computer Science")
print(student)
```

Since the key "major" does not exist, it is added with the specified default value as shown in the output below –

```
{'name': 'Alice', 'age': 21, 'major': 'Computer Science'}
```

## Add Dictionary Item Using the collections.defaultdict() Method

The collections.defaultdict() method in Python is a subclass of the built-in "dict" class that creates dictionaries with default values for keys that have not been set yet. It is part of the collections module in Python's standard library.

We can add dictionary items using the collections.defaultdict() method by specifying a default factory, which determines the default value for keys that have not been set yet. When accessing a missing key for the first time, the default factory is called to create a default value, and this value is inserted into the dictionary.

### Example

In this example, we are initializing instances of defaultdict with different default factories: int to initialize missing keys with 0, list to initialize missing keys with an empty list, and a custom function default_value to initialize missing keys with the return value of the function –

```
from collections import defaultdict
# Using int as the default factory to initialize missing keys with 0
d = defaultdict(int)
# Incrementing the value for key 'a'
d["a"] += 1
print(d)

# Using list as the default factory to initialize missing keys with an empty
list
d = defaultdict(list)
# Appending to the list for key 'b'
d["b"].append(1)
print(d)

# Using a custom function as the default factory
def default_value():
    return "N/A"
```

```
d = defaultdict(default_value)
print(d["c"])
```

The output obtained is as follows –

```
defaultdict(<class 'int'>, {'a': 1})
defaultdict(<class 'list'>, {'b': [1]})
N/A
```

Unix Tutorial

## CERTIFICATIONS

Business Analytics Certification

Java & Spring Boot Advanced Certification

Data Science Advanced Certification

Cloud Computing And DevOps

Advanced Certification In Business Analytics

Artificial Intelligence And Machine Learning

DevOps Certification

Game Development Certification

Front-End Developer Certification

AWS Certification Training

Python Programming Certification

## COMPILERS & EDITORS

Online Java Compiler

Online Python Compiler

Online Go Compiler

Online C Compiler

Online C++ Compiler

Online C# Compiler

Online PHP Compiler

Online MATLAB Compiler



Online Html Editor